

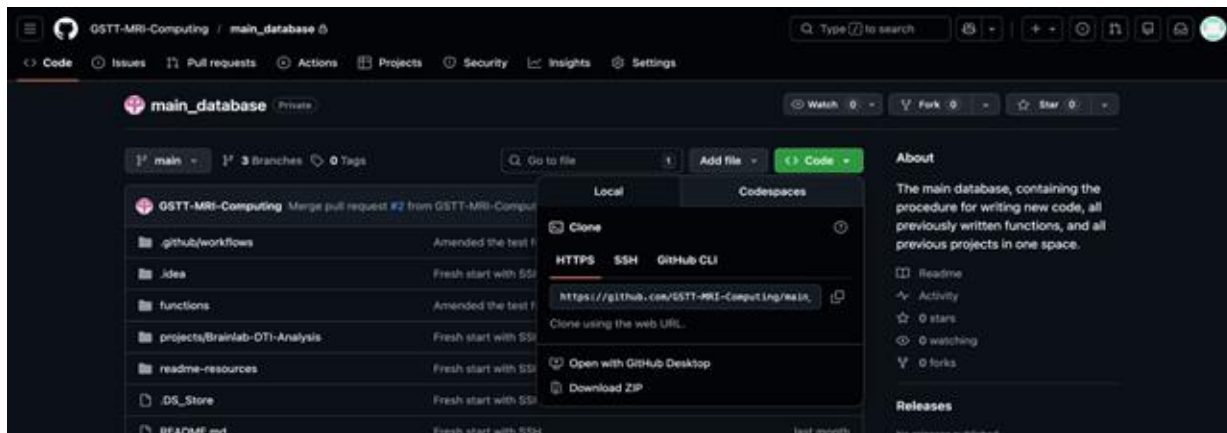
A guide to ...



with VS Code

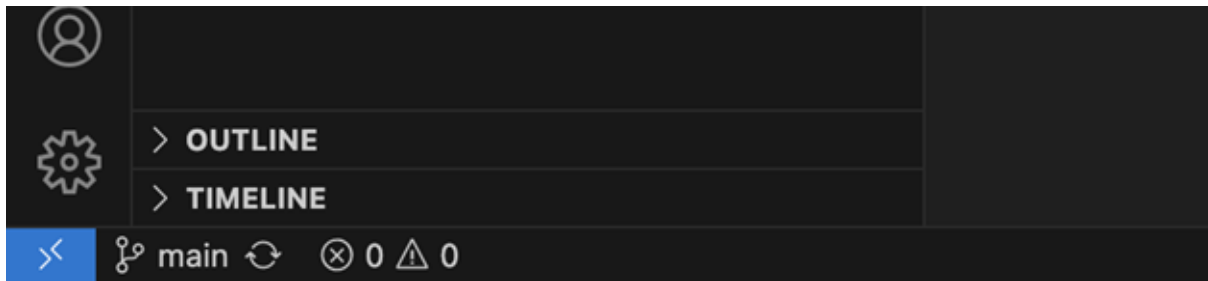
Setting Up Part 1

1. `git -version` (ensure you have GitHub downloaded)
2. <https://git-scm.com> (if not already downloaded)
3. Sign in to Co-Pilot with your GitHub account
4. Go to the page of the repo you want to download



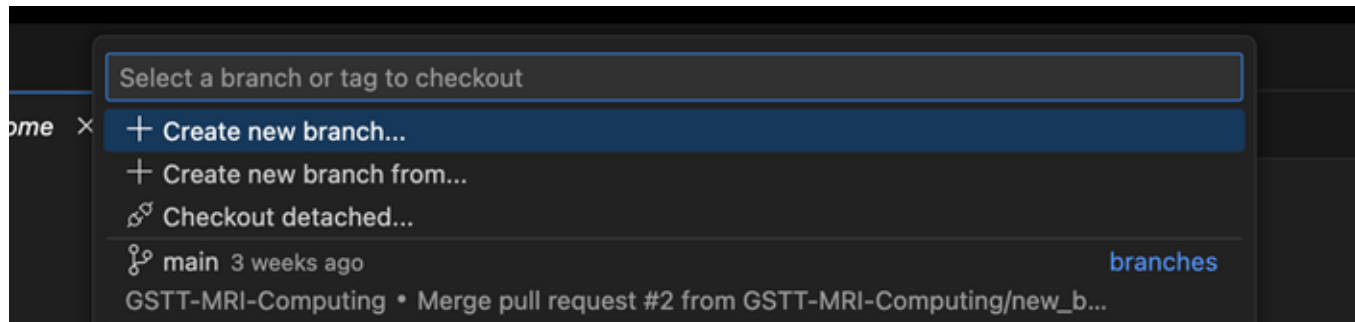
Setting Up Part 2

1. `Cmd + Shift + P`
2. `Git: Clone`
3. Paste repo
4. Follow authorisation steps (they pop up)
- 5.



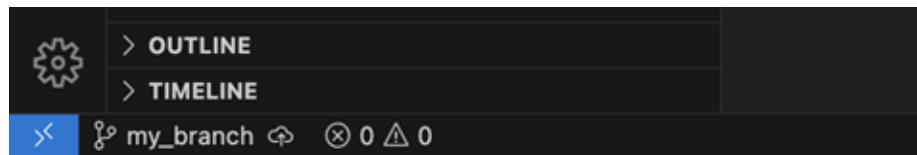
Setting Up Part 3

1.



2. Type in the name you want for your branch.

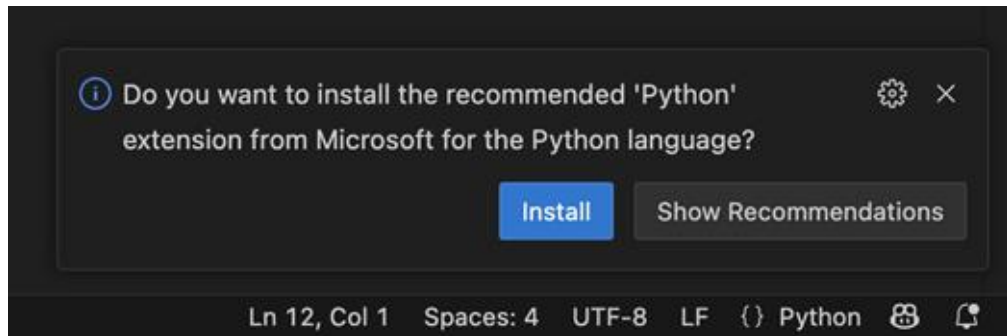
3. This should now look like this:



Setting up Python

1. Click on a Python File (which you make by right clicking)

2.



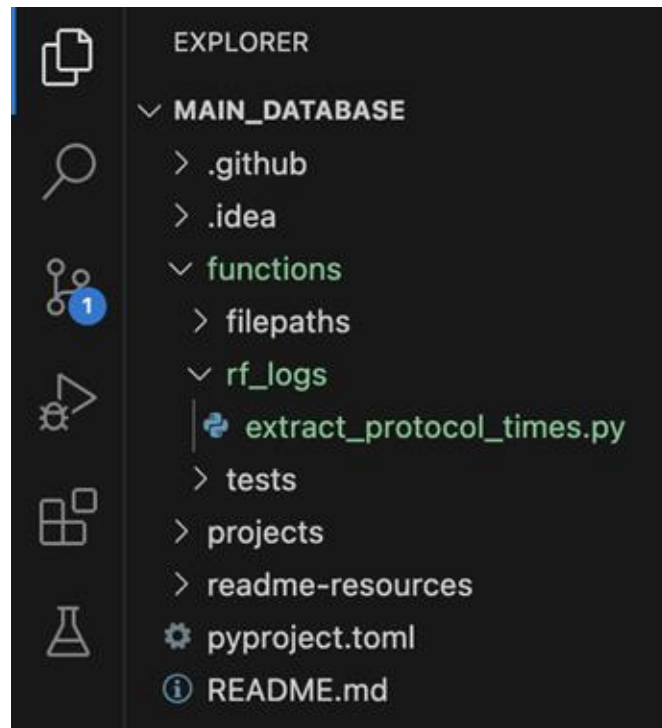
3. Press install

4. You should now see a play button at the top of your editor.

Naming convention

1. Find the appropriate area to create a new function. If there is not a category yet, create a directory.

In the example case, I have just created a folder called `rf_logs`, with a new python function called `extract_protocol_times.py`



Activating your virtual environment

1. `python3 -m venv .venv`
2. `source .venv/bin/activate`
3. `pip install ".[dev]"`
4. Cmd + Shift + P
5. Python: Select Interpreter
6. Start typing in `.venv/bin` and then click on the interpreter you are using in your venv.
7. Write any new dependencies in the `.toml` file

You can now write code :) Apologies for the length

```
[build-system]
requires = ["setuptools>=61.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "main_database"
version = "0.1.0"
dependencies = [
    "pydicom==2.4.4"
]

[project.optional-dependencies]
dev = ["pytest"]

[tool.setuptools.packages.find]
where = ["."]
include = ["functions*", "projects*"]

[tool.pytest.ini_options]
testpaths = ["functions/tests"]
python_files = ["test_*.py"]
```

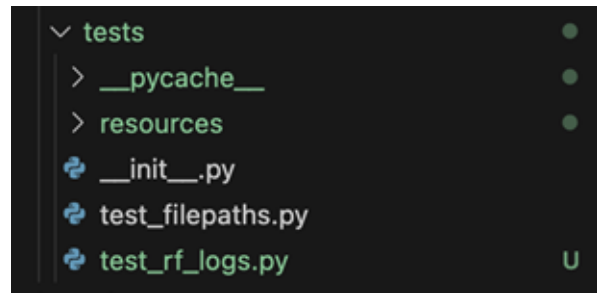
Making your own functions part 1

1. Once you have developed your new function (naming and location info in previous slide), you need to add it to be tested. Go into the test folder and add a file called

`test_<my_function>.py`

Use the templates provided in the GitHub which are downloaded locally in the first few steps

2. Add any files in the resources folder that you need to test your function, (for example if it needs a text file to see what the header is)



Making your own functions part 2

1. Now run:

```
pytest functions/tests/test_<my_function>.py
```

2. You should now see this. This means your function is ready for upload.

```
functions/tests/test_rf_logs.py . [100%]  
===== 1 passed in 0.05s =====
```

Uploading to GitHub

1. `git add .` (optionally add `git reset folder_to_exclude/`)
2. `git commit -m "Description of what you changed"`
3. `git push origin your_branch`

At this point, the tests will automatically run on GitHub and the moderator will either merge your code or send it back to you for amendments. If it is sent back, modify it and then complete the same three above steps.

Finishing up

Occasionally, you may have accidentally modified your code by accident after doing a pull request. If this happens, you will get this error:

```
error: Your local changes to the following files would be
overwritten by checkout:
```

```
functions/rf_logs/my_function.py
```

Please commit your changes or stash them before you switch branches.

If you have done this on purpose, re push. If not, just run:

Cleaning up

28. `git checkout main` (go back to the main branch)

29. `git branch -d your_branch`

30. `git pull origin main`

You have successfully created, tested and uploaded code 😊

